BOOTCAMP II ? ENTREGA INICIAL ? FASE 1

# Planejamento L?gico TaskTracker

Documento de arquitetura e receita l?gica para orientar a implementa??o em Python.

**Identifica??o acad?mica**
Nome completo: _____
Curso/turma: _____ Polo: _____
E-mail institucional: _____

**Sobre este documento:** este planejamento foi elaborado agora para apoiar a realiza??o das Fases 1 e 2. A Fase 1 n?o havia sido realizada anteriormente; este arquivo n?o afirma entrega, aprova??o ou avalia??o acad?mica pr?via.

### Prop?sito do documento

O enunciado da Fase 1 pede um documento de arquitetura que apresente o problema, os dados e as regras, a receita l?gica em portugu?s, o mapeamento para Git/GitHub e o link do v?deo. Este planejamento tamb?m foi alinhado aos requisitos t?cnicos da Fase 2, que implementar? o prot?tipo descrito aqui.

## 1. Identifica??o e entendimento do problema

### Contexto e problema

Na rotina de estudos, trabalho e atividades pessoais, ? comum acumular tarefas sem um lugar simples para registrar o que precisa ser feito. Anota??es espalhadas podem dificultar a consulta, a defini??o de prioridades e o acompanhamento dos prazos. Uma tarefa esquecida ou sem informa??o clara pode atrasar outras atividades.

### Objetivo e solu??o proposta

O TaskTracker ser? um programa de linha de comando para cadastrar tarefas e consultar a lista durante a execu??o. Para cada tarefa, o usu?rio informar? t?tulo, descri??o, prioridade e data limite. O sistema verificar? se o t?tulo foi preenchido e se a prioridade pertence ?s op??es permitidas. Cada novo registro come?ar? com o status ?Pendente?.

### P?blico e limites do prot?tipo

O prot?tipo poder? ser utilizado por estudantes e por qualquer pessoa que queira organizar tarefas de forma direta no terminal. Na Fase 2, os registros ficar?o em mem?ria enquanto o programa estiver aberto; n?o haver? banco de dados, grava??o permanente, edi??o ou exclus?o de tarefas. O objetivo ? praticar decomposi??o do problema, l?gica, Python e versionamento.

## 2. Decomposi??o do problema, dados e regras

### Entradas

| Dado | Como ser? utilizado |
|---|---|
| T?tulo | Identifica a tarefa. ? obrigat?rio e ser? armazenado sem espa?os extras nas extremidades. |
| Descri??o | Texto livre para detalhar o que precisa ser feito; poder? ficar vazio. |
| Prioridade | Indica a prioridade escolhida: Alta, M?dia/Media ou Baixa. O valor ser? padronizado ao armazenar. |
| Data limite | Texto informado pelo usu?rio para registrar a data planejada; n?o ter? valida??o avan?ada. |

### Sa?das

- Confirma??o quando uma tarefa for cadastrada.
- Mensagens de erro e novo pedido de entrada quando o t?tulo ou a prioridade forem inv?lidos.
- Lista organizada com todas as propriedades das tarefas cadastradas.
- Mensagem ?Nenhuma tarefa cadastrada no momento.? quando a lista estiver vazia.

### Regras do sistema

| Regra | Comportamento previsto |
|---|---|
| T?tulo obrigat?rio | Rejeitar texto vazio ou composto somente de espa?os, usando a remo??o de espa?os das extremidades antes de verificar. |
| Prioridade restrita | Aceitar somente Alta, M?dia, Media ou Baixa, sem diferenciar mai?sculas e min?sculas; repetir a pergunta em caso de erro. |
| Descri??o e data | Descri??o ? livre e data limite ? armazenada como texto, sem regras adicionais de formato. |
| Status inicial | Definir automaticamente o status como ?Pendente? no cadastro. |
| Estrutura em mem?ria | Guardar cada tarefa em um dicion?rio e todos os dicion?rios em uma lista. |

O dicion?rio de cada tarefa ter? as chaves `titulo`, `descricao`, `prioridade`, `data_limite` e `status`. A lista ser? criada ao iniciar o programa; por isso, os dados n?o ser?o preservados ap?s o encerramento.

## 3. Receita l?gica ? algoritmo em portugu?s

1. Iniciar o programa.
2. Criar uma lista vazia chamada tarefas, que guardar? os registros durante a execu??o.
3. Exibir o menu com as op??es cadastrar tarefa, visualizar tarefas e sair.
4. Ler a op??o escolhida pelo usu?rio.
5. Se a op??o for cadastrar, pedir o t?tulo e remover espa?os das extremidades para valid?-lo.
6. Enquanto o t?tulo estiver vazio ap?s essa remo??o, mostrar uma mensagem de erro e pedir outro t?tulo.
7. Solicitar a descri??o da tarefa.
8. Solicitar a prioridade. Comparar sem diferenciar mai?sculas e min?sculas com Alta, M?dia, Media e Baixa.
9. Enquanto a prioridade n?o corresponder a uma das op??es aceitas, mostrar uma mensagem de erro e pedir novamente. Guardar o valor em uma forma padronizada.
10. Solicitar a data limite e guard?-la como texto, sem valida??o avan?ada.
11. Criar um dicion?rio com t?tulo, descri??o, prioridade, data limite e status ?Pendente?.
12. Adicionar o dicion?rio ? lista de tarefas e confirmar o cadastro.
13. Se a op??o for visualizar, verificar se a lista est? vazia. Se estiver, informar que n?o h? tarefas; caso contr?rio, percorrer a lista e exibir todas as propriedades de cada tarefa.
14. Se a op??o n?o for cadastrar, visualizar nem sair, informar que a op??o ? inv?lida.
15. Ap?s cadastrar, listar ou informar uma op??o inv?lida, voltar ao menu. Encerrar somente quando o usu?rio escolher sair.

**Representa??o de um registro:**

```
tarefa = {
    "titulo": titulo,
    "descricao": descricao,
    "prioridade": prioridade,
    "data_limite": data_limite,
    "status": "Pendente"
}
```

# 4. Mapeamento arquitetural ? Git e GitHub

**Git** ? o sistema de controle de vers?o usado localmente para registrar mudan?as ao longo do desenvolvimento. Cada commit guarda um conjunto coerente de altera??es com uma mensagem que explica seu prop?sito. Isso permite acompanhar a evolu??o e consultar vers?es anteriores.

**GitHub** hospeda remotamente o reposit?rio Git, permitindo publicar o c?digo e os documentos. O hist?rico ser? enviado para um reposit?rio remoto na branch principal `main`. A publica??o s? ser? feita ap?s confirma??o do estudante.

### Estrutura planejada para a Fase 2

```
tasktracker-projeto/
??? README.md
??? .gitignore
??? docs/
?   ??? planejamento_logico.pdf
??? src/
    ??? main.py
```

O README apresentar? a finalidade do projeto e instru??es de execu??o; o .gitignore excluir? arquivos tempor?rios e ambientes virtuais; o PDF manter? este planejamento junto ao projeto; e `src/main.py` conter? a aplica??o CLI em Python.

### Evolu??o prevista dos commits

1. Registrar estrutura inicial, README, regras do .gitignore e planejamento.
2. Adicionar o menu principal e a estrutura inicial do programa.
3. Adicionar cadastro, valida??es e listagem.
4. Revisar a implementa??o e completar a documenta??o.

Ap?s os commits locais, ser? conferido o estado do reposit?rio, a branch e o hist?rico. Com autoriza??o, o reposit?rio ser? criado como p?blico e a branch `main` ser? enviada ao GitHub. Arquivos de refer?ncia e credenciais n?o far?o parte do reposit?rio.

# 5. Link da apresenta??o em v?deo

**Link do v?deo: PENDENTE ? preencher ap?s a grava??o**

O enunciado da Fase 1 prev? um pitch de 5 minutos publicado no YouTube, em formato horizontal (16:9), com participa??o do estudante em c?mera e ?udio claro. O v?deo deve estar como P?blico ou N?o Listado; o formato Shorts/vertical, v?deo privado e grava??o apenas da tela sem o rosto n?o atendem ?s instru??es. A Fase 2 tamb?m exige voz pr?pria durante a explica??o.

## Apresenta??o prevista

O estudante poder? apresentar sua identifica??o acad?mica, explicar a dificuldade que o TaskTracker pretende resolver, mostrar este documento, percorrer a receita l?gica e comentar a estrutura planejada para GitHub. O v?deo ainda n?o foi gravado nem publicado.

## Rela??o entre as fases

| Fase | Continuidade do projeto |
|---|---|
| Fase 1 ? Planejamento l?gico (10 pontos) | Definir o problema, os dados e as regras, escrever o algoritmo, planejar a estrutura Git/GitHub e preparar a apresenta??o. |
| Fase 2 ? Entrega intermedi?ria (15 pontos) | Usar este planejamento como refer?ncia, criar o reposit?rio, implementar a CLI em Python, validar as regras, registrar commits e demonstrar o c?digo. |
| Fase 3 ? Entrega final (25 pontos) | Etapa futura, fora do escopo deste trabalho. Docker e deploy n?o ser?o implementados agora. |

Este documento foi elaborado como planejamento para orientar o desenvolvimento. A entrega acad?mica e sua avalia??o dependem da submiss?o e dos crit?rios do curso.